

Paragliding Intelligence Platform

Volume 05 - Weather Engine Design v2 Technical

Developer-grade architecture for a paragliding-first meteo platform better than generic weather maps

Date: 2026-06-04

Scope Note: This volume replaces the high-level placeholder weather description with an implementation-ready weather engine design. It is written as a technical specification for backend, GIS, data engineering, ML, and mobile teams. The system is not intended to certify flight safety; it provides decision support with explicit uncertainty, hard blockers, and human pilot responsibility.

Table of Contents

1. 1. Purpose and competitive thesis
2. 2. Weather-engine principles
3. 3. Source strategy and provider matrix
4. 4. Target weather variables and units
5. 5. Weather data domain model
6. 6. Model-run lifecycle
7. 7. Ingestion architecture
8. 8. GRIB/NetCDF/Zarr processing pipeline
9. 9. Normalization and forecast cube design
10. 10. Spatial interpolation and vertical levels
11. 11. Terrain correction and microscale post-processing
12. 12. Paragliding-derived variables
13. 13. Nowcasting and live reality layer
14. 14. Forecast validation and model skill
15. 15. Site forecast generation
16. 16. Map tile and animation outputs
17. 17. APIs and internal service contracts
18. 18. Queue/event architecture
19. 19. Storage architecture
20. 20. Caching and performance
21. 21. Data quality rules
22. 22. Safety-critical weather blockers
23. 23. Monitoring and SLOs
24. 24. Security, licensing, and attribution
25. 25. Deployment topology
26. 26. MVP implementation slice
27. 27. V1/V2 roadmap
28. 28. Engineering backlog
29. 29. References

1. Purpose and competitive thesis

The Weather Engine is the core differentiator of the Paragliding Intelligence Platform. Generic weather applications show model layers, wind particles, cloud cover, rain and radar. A paragliding-first system must transform raw meteorology into launch-specific, altitude-aware, terrain-aware and pilot-aware decisions. The engineering goal is to turn forecast data into structured answers: when a launch is usable, when conditions degrade, which hazards dominate, and how confident the system is.

- Primary output: launch-hour forecast packs, flyability inputs, risk-engine feature vectors, XC potential fields and map visualization layers.
- Primary differentiator: site-specific correction loops using observed conditions, pilot reports, local stations, IGC tracks and forecast-error history.
- Primary safety rule: all recommendations must expose uncertainty, hard blockers and reasons; the system must never hide storm, gust, rotor, Foehn or airspace concerns behind a high score.
- Primary product target: answer “where can I fly, when should I launch, and what can go wrong?” faster than Windy/Ventusky-style generic layer inspection.

Capability	Generic weather map behavior	Next-edge paragliding weather engine behavior
Wind display	Show wind layer at selected altitude.	Convert wind to launch orientation fit, gust spread, altitude shear, drift risk and pilot-specific warnings.
Cloudbase	Show cloud and temperature variables.	Estimate usable cloudbase above launch/landing, XC ceiling, valley clearance and cloud-suck risk.
Thermals	Usually limited or indirect.	Compute thermal potential from radiation, terrain aspect, cloud cover, lapse rate, humidity and historical flight climbs.
Forecast confidence	Often implicit.	Compare models, observations and recent site errors; output confidence per site-hour and feature.
Safety	User manually interprets layers.	Hard blockers and explanations: storm growth, gust limit, Foehn, rotor exposure, rain, low cloud, visibility and severe shear.

2. Weather-engine principles

- Safety-first outputs: dangerous weather conditions are blockers, not mere negative weights in a score.
- Model-agnostic design: every provider is normalized into a canonical forecast cube, so the product can switch providers or add premium models without changing downstream features.
- Site-first downstream processing: raw grid forecasts are not shown directly to pilots until they are converted into site forecast packets and risk vectors.
- Explainable derived variables: every derived value must be traceable to input variables, model run, interpolation method and correction version.
- Freshness-aware caching: a stale safe forecast is worse than a missing forecast; stale data must visibly degrade confidence.
- Commercial tiering: MVP can use forecast APIs; PRO+/enterprise can use raw GRIB/NetCDF/Zarr ingestion, ensembles, nowcasting and digital twin correction.
- Auditability: every GO/MAYBE/NO-GO decision must be reproducible from stored inputs and algorithm versions.

Canonical principle:

raw_provider_data -> normalized_forecast_cube -> derived_weather_features -> site_forecast_pack -> risk/flyability/AI outputs

Forbidden architecture:

raw_provider_data -> LLM summary -> pilot recommendation

3. Source strategy and provider matrix

The weather engine uses a staged source strategy. The MVP depends on APIs that return ready-to-use JSON forecasts. V1 introduces raw model ingestion and local station/pilot observations. V2 adds ensemble uncertainty, nowcasting, satellite/radar processing and site digital twin correction. The design keeps raw provider terms, attribution and rate limits isolated in provider adapters.

Source	Role	Format	Update pattern	Tier	Implementation note
Open-Meteo	MVP multi-model forecast and historical backfill	JSON API	Provider-managed	Free/PRO	Fastest way to launch site forecasts and compare model families.
MET Norway Locationforecast	Nordic point forecasts	JSON	Provider-managed	Free/PRO	Good initial Norway support; not suitable for above-ground aviation use according to provider docs.
MET Norway Nowcast	Short-range precipitation nowcasting for Norway	JSON	Minutes	PRO+	Use as rain/storm hard blocker near launches.
MET Norway IsobaricGRIB	Aviation-style upper-air fields where available	GRIB	Model cycle	PRO+	Use for wind aloft and pressure-level data when applicable.
ECMWF Open Data	Global high-quality medium range model source	GRIB	Model cycle	PRO+	Primary premium source for European and global forecast cubes.
NOAA GFS via NOMADS	Global backup and comparison model	GRIB2	Model cycle	PRO+	Useful for model disagreement and long-range awareness.
Local stations	Reality layer and site correction	JSON/MQTT/CSV	1-10 min	PRO+/Club	Launch/landing stations become most valuable data source for correction.
Pilot reports	Human reality validation	Mobile API	Real time	All tiers	Moderated observations with reputation and anomaly checks.
IGC/track logs	Historical climbs and route outcomes	IGC/GPX	Post-flight	PRO+	Training data for thermal and route models.
Webcams	Cloud visibility and wind sock inference	Image stream	1-10 min	Club/PRO+	Computer vision supports launch reality layer; never sole safety input.
Radar/lightning/satellite	Nowcasting hazards	Raster/API	Minutes	PRO+	Thunderstorm, rain and overdevelopment warnings.

Licensing: Provider adapters must keep license metadata with every model_run and derived output. User-facing screens and exported reports must show required attribution for data sources.

4. Target weather variables and units

The platform must standardize all provider-specific names into a canonical variable catalog. Units must be stored explicitly. Direction must be derived from U/V components where possible instead of trusting provider direction strings. Values must preserve model run time, valid time, level, provider and processing version.

Canonical variable	Unit	Level	Used by	Notes
u_wind	m/s	10m, 100m,	wind, shear, drift, maps	Eastward wind

		pressure/height levels		component; convert to speed/direction.
v_wind	m/s	10m, 100m, pressure/height levels	wind, shear, drift, maps	Northward wind component.
wind_speed	m/s and km/h derived	all supported levels	flyability, map	Derived as $\sqrt{u^2 + v^2}$.
wind_direction	deg meteorological	all supported levels	launch fit	Direction wind comes from.
wind_gust	m/s	surface	hard blocker	Critical for launch safety.
temperature	K or C normalized	2m, pressure levels	lapse, cloudbase	Store canonical Kelvin internally; expose C.
dewpoint	K or C normalized	2m	cloudbase	Cloudbase estimation input.
relative_humidity	%	surface/levels	cloud and convection	Use for cloudbase/cloud layer checks.
pressure_msl	Pa/hPa	surface	trend, Foehn	Pressure gradient features.
precipitation_rate	mm/h	surface	rain blocker	Nowcast preferred where available.
total_cloud_cover	%	column	thermal/cloudbase	Cloud shading affects thermal potential.
low_mid_high_cloud	%	layers	cloud and visibility	Layered cloud helps distinguish high cloud from low launch-obscuring cloud.
cape	J/kg	column	storm risk	High CAPE with triggers increases danger.
cin	J/kg	column	storm risk	Can suppress convection; use carefully.
shortwave_radiation	W/m2 or J/m2	surface	thermal	Input to thermal trigger model.
boundary_layer_height	m	column	thermal/XC	Proxy for mixing depth when available.
visibility	m/km	surface	safety	Important for launch/landing and airspace awareness.
orography	m	static	terrain correction	May come from provider or DEM.

Wind conversion:

$$\text{wind_speed_mps} = \sqrt{u_wind^2 + v_wind^2}$$

$$\text{wind_speed_kmh} = \text{wind_speed_mps} * 3.6$$

$$\text{wind_direction_deg} = (\text{atan2}(-u_wind, -v_wind) * 180/\pi) \bmod 360$$

Gust spread:

$$\text{gust_spread_kmh} = \text{wind_gust_kmh} - \text{wind_speed_10m_kmh}$$

5. Weather data domain model

The Weather Engine separates raw data, normalized data, derived data and user-facing site outputs. This prevents accidental mixing of incompatible levels and makes recommendations reproducible.

weather_source

id, name, provider_type, license, attribution, status

model_run

id, source_id, model_name, run_time_utc, cycle, forecast_horizon_hours,
raw_uri, manifest_hash, ingestion_status, processor_version

forecast_cube

id, model_run_id, variable, level_type, level_value, grid_id,
storage_uri, valid_from_utc, valid_to_utc, units, normalization_version

Paragliding Intelligence Platform - Volume 05 Weather Engine Design v2 Technical
derived_weather_field

id, model_run_id, derived_name, algorithm_version, grid_id,
storage_uri, valid_time_utc, confidence, source_variables

site_forecast_pack

id, site_id, model_run_id, valid_time_utc, corrected_version,
wind_json, thermal_json, cloudbase_json, risk_features_json, quality_flags_json

Layer	Storage	Retention	Read path	Why
Raw provider files	Object storage: MinIO/S3	30-180 days by tier	Reprocessing only	Audit and replay.
Normalized cubes	Zarr/NetCDF + metadata DB	30-365 days	Processing and map generation	Efficient multidimensional slicing.
Derived fields	Zarr/COG/tiles + metadata DB	30-365 days	Maps, risk engine	Thermal, cloudbase, shear, storm risk.
Site forecast packs	PostgreSQL/ TimescaleDB	1-5 years	APIs and app	Fast per-site UX and audit.
Observations	TimescaleDB	Long-term	Correction and validation	Learning loop.

6. Model-run lifecycle

- 30. Discover available provider run: check provider manifest, directory listing, API timestamp or scheduled cycle time.
- 31. Create model_run row with status DISCOVERED and immutable run_time_utc/cycle metadata.
- 32. Fetch raw files to object storage using idempotent paths and checksum validation.
- 33. Validate file completeness, required variables, expected time steps and basic geospatial metadata.
- 34. Normalize raw data into canonical cubes with consistent coordinates, units, variable names and level metadata.
- 35. Derive paragliding variables: altitude winds, gust spread, launch direction fit inputs, thermal potential, cloudbase, storm indicators, precipitation nowcast blockers.
- 36. Generate site forecast packs for each launch and configured forecast horizon.
- 37. Publish events for cache warming, map tile generation, flyability scoring, AI briefing context generation and alert evaluation.
- 38. Mark model_run READY only when essential variables and site packs pass quality checks. If degraded, mark PARTIAL and expose quality flags.

Statuses:

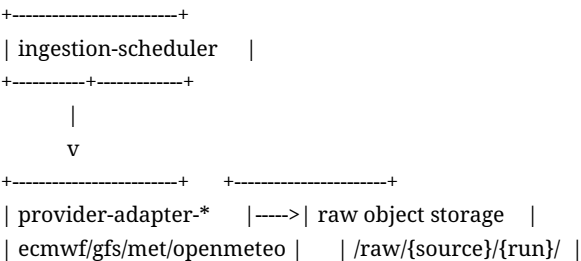
DISCOVERED -> FETCHING -> FETCHED -> VALIDATING -> NORMALIZING -> DERIVING -> SITE_PACKS -> READY
DISCOVERED -> FETCHING -> FAILED_FETCH
VALIDATING -> FAILED_VALIDATION
DERIVING -> PARTIAL_READY

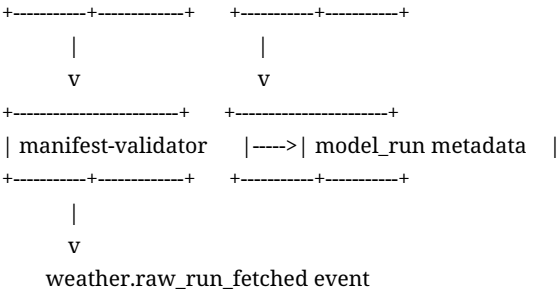
Idempotency key:

{source}:{model_name}:{cycle}:{run_time_utc}:{forecast_step}:{variable_group}

7. Ingestion architecture

Weather ingestion is implemented as provider adapters plus a common ingestion supervisor. Provider adapters know URL patterns, API parameters, authentication, attribution and provider-specific data quirks. The supervisor schedules jobs, enforces concurrency, stores manifests, validates checksums and emits events.





Component	Responsibility	Failure behavior
ingestion-scheduler	Starts cycles based on provider schedule and retries missing runs.	No direct provider logic; marks stale if expected run missing.
provider-adapter-ecmwf	Builds ECMWF file list, downloads GRIB, records attribution.	Retries per file; allows partial run only if essential variables exist.
provider-adapter-gfs	Builds NOMADS/GFS queries for needed variable groups.	Backoff on 429/5xx; optional fallback to previous cycle.
provider-adapter-metno	Fetches point forecasts/nowcasts for configured coordinates.	Requires compliant User-Agent and polite caching.
provider-adapter-openmeteo	MVP forecast retrieval and historical forecast backfill.	Caches responses, respects rate limits.
manifest-validator	Checks expected files, sizes, checksums, variable inventory.	Failed essential variables prevent READY status.
raw-file-indexer	Writes file metadata and object URIs to DB.	Never overwrites raw files; immutable paths.

Object storage layout:

```
/raw/{source}/{model}/{yyyyMMdd}/{cycle}/manifest.json
/raw/{source}/{model}/{yyyyMMdd}/{cycle}/grib/{variable_group}_{step}.grib2
/raw/{source}/{model}/{yyyyMMdd}/{cycle}/api/{lat}_{lon}_{hash}.json
/normalized/{source}/{model}/{yyyyMMdd}/{cycle}/zarr/{cube_name}.zarr
/derived/{source}/{model}/{yyyyMMdd}/{cycle}/{derived_name}.zarr
/tiles/{layer}/{yyyyMMdd}/{cycle}/{z}/{x}/{y}.pbf
```

8. GRIB/NetCDF/Zarr processing pipeline

Raw GRIB is not suitable for direct application queries. The engine converts GRIB into canonical, chunked data structures for efficient slicing by variable, time, altitude/pressure and geographic bounds. The recommended Python stack is ecCodes/cfgrib + xarray + Dask for GRIB, xarray + Zarr for normalized cubes, and GDAL/rasterio for geospatial rasters and tiles.

Pseudo pipeline:

```
raw_grib_files = discover_raw_files(model_run)
ds = open_mfdataset(raw_grib_files, engine='cfgrib', backend_kwargs={...})
ds = normalize_coordinates(ds) # longitude, latitude, time, step, level
ds = normalize_units(ds) # K, m/s, Pa, kg/m2/s, etc.
ds = rename_variables(ds, mapping) # provider-specific -> canonical
ds = subset_region(ds, bbox) # optional EU/Norway/Alps tiles
ds = rechunk_for_access(ds) # time/level/lat/lon chunks
ds.to_zarr(storage_uri, consolidated=True)
```

Processing step	Technical rule	Validation
Coordinate normalization	Use lat/lon WGS84 order; normalize longitudes to -180..180 or document 0..360 consistently.	Bounding box queries return correct points around dateline.
Time normalization	Store run_time_utc, forecast_step_hours and valid_time_utc separately.	No duplicate valid times for same model/run/step.
Level normalization	Represent 10m/100m as height_agl; pressure levels as hPa; derived site levels as meters_msl/agl.	Each variable declares supported level_type.
Unit normalization	Canonical internal units: m/s, K, Pa, kg/m2/s,	Unit conversion tests for each provider.

	W/m2 where source allows.	
Chunking	Chunk by time, level and spatial tiles for common access paths.	P95 site extraction under SLA.
Metadata	Write provider/source/license/algorithm version into cube attributes.	Every output can trace to source model run.

Critical Data Rule: The processing layer must never silently fill missing variables with zero. Missing values must become null/NaN plus quality flags. Zero wind and missing wind are completely different safety states.

9. Normalization and forecast cube design

The canonical forecast cube is the internal product of normalization. The cube is optimized for deriving site-level features and map layers. It is not a relational table for every grid cell; relational storage is used for metadata, site packs and time-series observations.

Canonical dimensions:
time: valid_time_utc
y: latitude index
x: longitude index
level: optional vertical level

Core attributes:
source_id, model_name, run_time_utc, forecast_step_hours,
variable, units, level_type, level_value, grid_resolution,
projection, bbox, normalization_version, provider_license

Recommended storage:
Zarr for normalized forecast cubes
Cloud Optimized GeoTIFF for some raster snapshots
PostgreSQL only for metadata and derived site packs

Access pattern	Best storage	Example
Point forecast for launch	Site forecast pack in PostgreSQL/TimescaleDB	GET /sites/{id}/forecast
Regional map animation	Pre-generated vector/raster tiles	Wind particles, thermal heatmap
Scientific reprocessing	Zarr normalized cube	Recompute thermal model v3
Forecast validation	TimescaleDB observations + site packs	Forecast error per site/hour
AI briefing context	PostgreSQL materialized context table	Grounded LLM prompt

10. Spatial interpolation and vertical levels

Paragliding decisions depend on launch altitude, ridge height, valley landing altitude and expected flight levels. The engine must derive weather at site-specific altitudes rather than showing only 10m surface wind.

Interpolation type	Use case	Default method	Quality flag
Horizontal point extraction	Launch forecast from grid	Bilinear interpolation for continuous fields; nearest for categorical quality flags.	distance_to_grid_center_km
Vertical interpolation	Wind/temperature at launch altitude and flight levels	Linear interpolation in height or log-pressure domain depending on available levels.	level_source: exact/interpolated/extrapolated
Time interpolation	Hourly site pack from 3h source steps	Linear for wind/temp, conservative for precipitation, no interpolation for lightning events.	time_interpolated_bool
Terrain adjustment	Correct grid elevation mismatch	Apply lapse-rate and exposure corrections where validated.	terrain_delta_m
Model blending	Combine models for confidence, not blindly average safety blockers.	Weighted ensemble only for derived confidence; hard blockers remain source-specific.	model_agreement_score

Site wind extraction levels:

```
launch_msl = launch.altitude_m
landing_msl = landing.altitude_m
ridge_msl = max_terrain_within_1km(site)
levels_msl = [launch_msl + 10, launch_msl + 100, launch_msl + 300,
              landing_msl + 200, ridge_msl + 100, 1000, 1500, 2000, 2500]
```

Quality rule:

```
if vertical interpolation requires extrapolation > 200m above/below available levels:
    set quality_flag = LEVEL_EXTRAPOLATED
    reduce confidence
```

11. Terrain correction and microscale post-processing

Weather models have grid cells much larger than individual launches. The engine uses terrain-derived features to estimate local effects, but must distinguish validated correction from experimental hints. Corrections are applied conservatively and logged.

Terrain feature	Source	Purpose	Implementation
Elevation delta	DEM vs model orography	Temperature and wind exposure correction	grid_elevation_m - launch_elevation_m
Slope angle	DEM	Thermal trigger and launch exposure	Compute within 100m/500m buffers.
Aspect	DEM	Sun-facing slope heating	Compare with solar azimuth/elevation.
Ridge exposure	DEM + wind direction	Rotor/wind acceleration risk	Upwind/downwind ridge profile and lee-side classification.
Valley axis	DEM hydrology/morphology	Valley wind channeling	Approximate valley orientation and pressure gradient.
Roughness/landcover	Optional landcover	Thermal and turbulence modifiers	Forest/urban/water/snow penalties.

Terrain correction boundary:

```
corrected_value = raw_value + deterministic_adjustment + ml_residual_correction
```

Where:

```
deterministic_adjustment: physically explainable, versioned, limited by caps
ml_residual_correction: site-trained correction with confidence and sample count
```

Never:

```
Apply hidden correction without exposing corrected_version and confidence.
```

12. Paragliding-derived variables

The Weather Engine publishes derived features consumed by the Flyability Engine, Risk Engine, XC Planner and AI Briefing service. These are not general meteorological variables; they are paragliding decision features.

Derived feature	Inputs	Output	Consumer
launch_wind_fit	wind direction/speed at launch, allowed site sectors	0..1 score plus explanation	Flyability
gust_spread_index	10m wind, gust, station gusts	LOW/MED/HIGH/CRITICAL	Risk
wind_shear_index	wind levels launch/500/1000/1500m	0..100 risk	Risk/XC
cloudbase_agl_estimate	temp, dewpoint, RH profile, cloud cover	meters AGL plus confidence	Flyability/XC
thermal_potential	radiation, lapse, cloud, terrain, historical climbs	m/s estimate + class	XC Planner

overdevelopment_risk	CAPE, CIN, humidity, radiation, radar/satellite trend	0..100 risk	Safety blocker
rotor_exposure	wind direction, speed, ridge/lee terrain	0..100 risk	Safety blocker
foehn_risk	cross-mountain pressure gradient, wind aloft, humidity contrast	LOW/MED/HIGH	Safety blocker
valley_wind_risk	valley axis, pressure gradient, heating, time of day	0..100 risk	Site forecast
xc_potential	thermal_potential, cloudbase, wind drift, landing options	distance class / score	XC Planner
model_disagreement	multi-model values per feature	0..100 uncertainty	All decision outputs

Example site-hour feature vector:

```
{
  "site_id": "site_123",
  "valid_time_utc": "2026-06-04T12:00:00Z",
  "wind_launch_kmh": 18.4,
  "wind_launch_dir_deg": 312,
  "gust_kmh": 27.0,
  "gust_spread_kmh": 8.6,
  "wind_1000m_kmh": 24.1,
  "shear_index": 38,
  "cloudbase_agl_m": 1280,
  "thermal_potential_ms": 2.7,
  "precip_probability": 0.05,
  "overdevelopment_risk": 22,
  "rotor_exposure": 31,
  "model_agreement_score": 0.82,
  "quality_flags": []
}
```

13. Nowcasting and live reality layer

The platform becomes better than generic weather maps by showing forecast-vs-reality. The live reality layer fuses stations, pilot reports, webcams, radar, lightning and recent track evidence. These observations do not replace forecasts; they override, validate or degrade confidence in the current site window.

Observation type	Ingestion path	Validation	Use
Launch station wind	station-ingest-service via API/MQTT	Range checks, stuck-sensor detection, spatial consistency	Correct current wind and gust blockers.
Landing station wind	station-ingest-service	Cross-check launch/valley station difference	Valley wind and landing risk.
Pilot report	mobile API	Reputation, proximity, time decay, outlier detection	Crowd reality layer and correction labels.
Webcam frame	camera-ingest-service	Timestamp and visibility checks	Cloud/visibility/windsock assistance.
Radar nowcast	radar-adapter	Provider freshness and georeference checks	Rain hard blocker.
Lightning strike	lightning-adapter	Distance/time decay	Storm hard blocker.
IGC/live track climbs	tracking-service	Flight validation and altitude sanity	Thermal validation and XC potential.

Reality override rule examples:

```
if station_gust_kmh > site.max_gust_kmh:
    current_status = NO_GO
    reason += "Observed gust exceeds launch limit"
```

```
if lightning_within_km <= 25 and strike_age_min <= 30:
```

```
current_status = NO_GO
reason += "Recent lightning near flying area"
```

```
if pilot_reports >= 3 and all report "strong rotor":
    increase rotor_exposure and require manual review flag
```

14. Forecast validation and model skill

The platform must measure forecast skill per site, model, variable, forecast horizon and weather regime. This drives confidence scores, premium model comparison and digital twin corrections.

Metric	Variable	Grouping	Purpose
MAE	wind speed, temperature, cloudbase	site/model/horizon/month	Basic correction and model selection.
Circular error	wind direction	site/model/horizon/wind sector	Launch sector reliability.
Brier score	rain/storm event	site/region/horizon	Probability calibration.
Hit/miss/false alarm	NO-GO blockers	site/model/season	Safety event validation.
Ranked model skill	feature vector	site/time-of-day/regime	Provider weighting.
Bias by wind sector	wind speed/gust	launch orientation	Site digital twin correction.

```
forecast_error_sample:
    forecast_value = site_forecast_pack.wind_launch_kmh
    observed_value = station_observation.wind_avg_kmh
    error = observed_value - forecast_value
    group_key = {site_id, model_name, horizon_hours, wind_sector, season, time_of_day}
```

Correction minimum sample rules:

- n < 30: do not apply ML correction, only show historical bias hint
- 30 <= n < 200: apply capped correction with low confidence
- n >= 200: allow model-specific correction with confidence interval

15. Site forecast generation

Site forecast generation converts forecast cubes into the API-ready packets used by the app. It must run after every model cycle and after relevant nowcast/observation updates.

```
for model_run in ready_model_runs:
    for site in active_sites:
        terrain = load_site_terrain_profile(site)
        for valid_time in model_run.valid_times:
            raw = extract_cube_values(site, valid_time, terrain.required_levels)
            derived = compute_derived_features(site, raw, terrain)
            corrected = apply_digital_twin_corrections(site, derived)
            quality = evaluate_quality_flags(raw, derived, corrected)
            write_site_forecast_pack(site, model_run, valid_time, corrected, quality)
            emit("weather.site_pack_ready", site_id, valid_time)
```

Site pack field	Type	Description
site_id	uuid	Launch site identifier.
valid_time_utc	timestamp	Forecast valid time.
source_model_runs	jsonb	Models/providers used.
wind	jsonb	Speed/direction/gust by launch and altitude levels.
cloudbase	jsonb	Estimated AGL/MSL cloudbase and confidence.
thermal	jsonb	Thermal potential, trigger likelihood and

		uncertainty.
precip_storm	jsonb	Rain, radar, lightning, CAPE/OD indicators.
terrain_risk	jsonb	Rotor/valley/Foehn/wave indicators.
model_agreement	jsonb	Disagreement and confidence by feature.
quality_flags	jsonb	Missing variables, stale data, interpolation warnings.
algorithm_versions	jsonb	Traceability for all derived outputs.

16. Map tile and animation outputs

The map layer engine produces both general weather layers and paragliding-specific decision layers. Generic wind particles are necessary for user trust, but the premium differentiator is site-specific and route-specific derived overlays.

Layer	Source	Tile type	Refresh	Tier
Wind particles 10m/500/1000/1500m	forecast_cube u/v wind	Vector/raster + WebGL field	model cycle	Free/PRO
Gust heatmap	derived gust fields	Raster tiles	model cycle/nowcast	Free/PRO
Cloudbase AGL	derived cloudbase	Raster tiles	model cycle	PRO
Thermal potential	derived thermal field	Raster/vector tiles	hourly/model cycle	PRO+
Rotor exposure	terrain + wind	Raster tiles	hourly	PRO+
Foehn risk zones	pressure gradient + wind aloft	Vector/raster	model cycle	PRO+
Storm nowcast	radar/lightning/satellite	Raster/vector	5-15 min	PRO+
Model disagreement	multi-model comparison	Raster	model cycle	PRO+
Launch status icons	site_forecast_pack	Vector points	hourly/current	All tiers

Tile generation event:

```
weather.derived_field_ready -> tile-worker
  input: derived_weather_field.storage_uri
  output: /tiles/{layer}/{run}/{z}/{x}/{y}.pbk or .png
  metadata: bounds, min_zoom, max_zoom, legend, units, algorithm_version
```

17. APIs and internal service contracts

External APIs are defined in Volume 04. This chapter defines Weather Engine internal contracts that the API Gateway consumes. These contracts are stable between services and should be represented in OpenAPI/AsyncAPI schemas.

GET /internal/weather/sites/{site_id}/pack?from=...&to=...&models=best

Response 200:

```
{
  "site_id": "...",
  "generated_at": "...",
  "hours": [
    {
      "valid_time_utc": "...",
      "best_model": "ecmwf-ifs",
      "wind": {"launch_kmh": 18, "dir_deg": 312, "gust_kmh": 27},
      "cloudbase": {"agl_m": 1280, "confidence": 0.74},
      "thermal": {"potential_ms": 2.7, "class": "good"},
      "risk_features": {"rotor": 31, "foehn": "low", "od": 22},
      "quality_flags": []
    }
  ]
}
```

POST /internal/weather/reprocess

Body:

```
{
  "model_run_id": "...",
  "scope": "site_packs|derived_fields|tiles",
  "algorithm_version": "thermal-v2.1.0",
  "site_ids": ["optional"]
}
```

Contract	Producer	Consumer	SLA
SiteForecastPack	weather-processing-service	api-gateway, flyability-engine, ai-briefing-service	P95 read < 150ms from cache.
DerivedWeatherField	weather-processing-service	tile-service, xc-planner	Available within model-cycle + 15-45 min depending provider.
WeatherQualityFlag	validator-service	all downstream services	No output may suppress quality flags.
ObservedWeatherSample	station/pilot/radar adapters	validation/digital twin/risk	Current samples available within 60s for stations.
ForecastErrorSample	validation-service	ml-correction-service	Generated daily/hourly depending source.

18. Queue/event architecture

The Weather Engine is event-driven. Weather runs can be large; synchronous processing would make the API fragile. The recommended implementation uses Kafka/Redpanda or RabbitMQ for events and Celery/RQ/Arq for worker execution in early MVP.

Topics / queues:

```
weather.run_discovered
weather.raw_file_fetched
weather.run_validated
weather.cube_normalized
weather.derived_field_ready
weather.site_pack_ready
weather.quality_degraded
observation.station_sample_received
observation.pilot_report_received
validation.forecast_error_computed
ml.correction_model_updated
alert.weather_window_changed
```

Event	Payload keys	Triggered by	Consumers
weather.run_discovered	source, model, run_time, cycle	scheduler	fetch workers
weather.raw_file_fetched	model_run_id, uri, checksum	provider adapter	validator
weather.cube_normalized	model_run_id, cube_uri, variables	normalizer	derived feature workers
weather.site_pack_ready	site_id, valid_time, model_run_id	site generator	cache, flyability, alerts, AI
weather.quality_degraded	scope, reason, severity	validator	ops, API, UI banners
observation.station_sample_received	station_id, sample_time, values	station ingest	nowcast, validation, risk
validation.forecast_error_computed	site_id, model, variable, error	validation job	ML correction

19. Storage architecture

Storage	Data	Why	Example path/table
Object storage	Raw GRIB/JSON, normalized Zarr, derived rasters	Cheap immutable data and reprocessing	/raw/ecmwf/ifs/20260604/00/...
PostgreSQL	Metadata, launch sites, algorithm versions, entitlements	Relational consistency	weather.model_runs

PostGIS	Sites, terrain zones, forecast tile bounds	Spatial queries and indexes	flying.launch_sites
TimescaleDB	Observations, site forecast packs, forecast errors	Time-series queries and retention	weather.site_forecast_packs
Redis	Current site forecast, active alerts, map metadata	Low-latency API responses	cache:site:{id}:current
Vector/raster tile cache	Map tiles	Fast map rendering	/tiles/thermal/{z}/{x}/{y}.pbf

Recommended SQL tables owned by Weather Engine:

```

weather.sources
weather.model_runs
weather.raw_files
weather.forecast_cubes
weather.derived_fields
weather.site_forecast_packs
weather.weather_quality_flags
weather.station_observations
weather.pilot_weather_reports
weather.forecast_error_samples
weather.correction_models
weather.tile_manifests

```

20. Caching and performance

The app must feel as fast as a consumer weather map even though the backend performs heavy scientific processing. Therefore, mobile/API paths must read precomputed site packs and tiles, not raw cubes.

Path	Cache key	TTL	Invalidation
Current site forecast	site:{id}:forecast:current	5-15 min	new site_pack or observation override
Hourly forecast list	site:{id}:forecast:{from}:{to}:{tier}	30-60 min	new model run
Map tile manifest	tiles:{layer}:{run}	until next run	new tile generation
Launch status points	region:{bbox}:launch_status:{hour}	5-15 min	site_pack_ready batch
AI briefing weather context	briefing_context:{site}:{window}	15-60 min	site pack/risk update
Station current samples	station:{id}:current	30-120 sec	new station sample

- External API requests should not directly query Zarr/GRIB; only internal batch jobs access heavy cubes.
- Use regional precomputation for launch-dense areas: Norway, Alps, Pyrenees, Balkans, Turkey, Himalaya priority zones.
- Cache warming should run for favorite sites and paid users after every model cycle.
- If a user pans a map, load launch status vector points first, then optional expensive raster layers.

21. Data quality rules

Rule ID	Rule	Severity	Downstream behavior
WQ-001	Required wind variables missing for site-hour.	CRITICAL	No GO recommendation; show unavailable/degraded.
WQ-002	Forecast model run older than freshness threshold.	HIGH	Degrade confidence; show stale data badge.
WQ-003	Surface wind from model disagrees with station by more than configured threshold.	MED/HIGH	Use reality overlay; create forecast error sample.
WQ-004	Vertical interpolation extrapolated beyond safe altitude gap.	MED	Degrade altitude-wind confidence.
WQ-005	Radar/lightning feed stale while storm risk > threshold.	HIGH	Do not downgrade storm blocker; show nowcast unavailable warning.
WQ-006	Provider API terms/attribution missing.	HIGH	Block public output for affected source until metadata fixed.

WQ-007	Observation sensor appears stuck or impossible.	MED	Exclude from corrections; flag station health.
WQ-008	Model disagreement critical for wind/gust/storm.	HIGH	Prevent strong GO output; require MAYBE/NO-GO explanation.

Quality flag propagation:

raw validation flag -> forecast cube metadata -> derived field metadata -> site_forecast_pack.quality_flags -> API/UI/AI briefing

AI rule:

LLM briefing must include quality flags in context and must not convert degraded data into confident recommendations.

22. Safety-critical weather blockers

The Weather Engine does not decide pilot action alone, but it must publish hard blocker inputs that downstream services cannot ignore. These blockers can force NO-GO or at minimum prevent positive recommendation language.

Blocker	Input	Example threshold behavior	Output
Observed gust above site limit	station gust or nowcast gust	<code>gust_kmh > site.max_gust_kmh</code>	NO_GO_CURRENT
Forecast gust above limit	model gust	future gust above site threshold	NO_GO_FORECAST_HOUR
Thunderstorm/lightning proximity	lightning/radar/CAPE	strike within configured distance/time	NO_GO_STORM
Rain at launch/route	radar/forecast precip	precip intensity above threshold	NO_GO_RAIN
Low cloud/visibility	cloudbase/visibility/webcam	cloudbase below safe AGL or launch obscured	NO_GO_VISIBILITY
Foehn/wave risk high	pressure gradient, wind aloft, humidity contrast	high mountain wind regime	NO_GO_MOUNTAIN_FLOW
Rotor exposure critical	terrain + wind speed/direction	lee side strong wind	NO_GO_ROTOR
Model disagreement high	multi-model comparison	safety-critical variables diverge	MAYBE_UNCERTAIN

Threshold Design: Thresholds must be site-configurable. A beginner training hill, high alpine XC launch, tandem operation and competition task should not share identical thresholds.

23. Monitoring and SLOs

SLO	Target	Measurement
Provider run ingestion freshness	95% of expected runs ingested within provider cycle + 45 min	model_run status timestamps
Site forecast availability	99% for active paid-region sites	site_pack coverage by horizon
Current observation latency	P95 station sample visible in < 60 sec	station ingest to cache
API forecast read latency	P95 < 250 ms cached	gateway metrics
Tile availability	P95 map tiles available within run + 60 min	tile manifest completeness
Forecast validation daily job	99% successful daily	validation run status
Quality flag propagation	100% critical flags visible to API	contract test
No stale-safe output	0 incidents where stale data displayed as fresh	audit rule

Metrics:

```

weather_ingestion_runs_total{source,status}
weather_run_latency_seconds{source,model}
weather_cube_normalization_seconds{model,region}
weather_site_pack_generated_total{model,site_region}
weather_site_pack_missing_total{reason}
weather_quality_flags_total{severity,rule_id}
weather_forecast_error_mae{site,model,variable,horizon}
weather_station_sample_latency_seconds{station_id}

```

24. Security, licensing, and attribution

- Provider API keys and credentials are stored in a secrets manager, never in app clients or repository config.
- Weather file downloads are outbound-only from ingestion workers; raw provider URLs are not exposed to users.
- Every source record stores license, attribution, allowed redistribution flag and commercial-use notes.
- Public API responses include source attribution where required by provider terms.
- Paid-tier enforcement happens at API Gateway and tile layer manifest generation; premium source data must not leak through free endpoints.
- Station data ownership is explicit: club station owners can choose public, club-only or private data sharing.
- Observation manipulation protection: reputation, anomaly detection, moderation and anti-spam apply to pilot reports before they influence scoring.

Source metadata required fields:

```
{
  "source_id": "ecmwf-open-data",
  "license": "CC-BY-4.0 / provider terms",
  "attribution_text": "Contains modified ECMWF data ...",
  "commercial_use_allowed": true,
  "redistribution_allowed": true,
  "requires_user_agent": false,
  "requires_api_key": false,
  "terms_url": "stored in admin metadata"
}
```

25. Deployment topology

Kubernetes namespace: weather

Deployments:

```
weather-ingestion-scheduler
provider-adapter-openmeteo
provider-adapter-metno
provider-adapter-ecmwf
provider-adapter-gfs
weather-validator
forecast-normalizer
derived-feature-worker
site-pack-generator
tile-generator
observation-ingest-service
forecast-validation-service
correction-model-service
```

Stateful/managed dependencies:

```
PostgreSQL + PostGIS + TimescaleDB
Redis
Object storage (MinIO/S3)
Queue (Kafka/Redpanda/RabbitMQ)
Prometheus/Grafana/Loki
```

Workload	CPU/RAM profile	Scaling rule
API read service	Low CPU, memory cache	Horizontal by RPS.
Provider fetchers	Network I/O	Scale by provider/run/file count with concurrency limits.
GRIB normalizer	CPU/RAM heavy	Scale with Dask workers / batch nodes.
Derived feature workers	CPU moderate/high	Scale by region/time horizon.

Tile generator	CPU/GPU optional	Scale by layer and zoom queue depth.
Station ingest	Low CPU, low latency	Horizontal by station count.
Validation/ML jobs	Batch CPU/RAM	Scheduled, spot/preemptible if cloud.

26. MVP implementation slice

The MVP should avoid heavy GRIB processing at first while keeping the architecture compatible with raw model ingestion. This allows launch with useful product value and later upgrade without rewriting downstream APIs.

MVP component	Implementation
Forecast source	Open-Meteo API for global multi-model basics; MET Norway Locationforecast/Nowcast for Norway where applicable.
Site coverage	Manual launch database for Norway/selected Alps, with allowed wind directions and thresholds.
Site packs	Hourly site forecast packs generated from API point forecasts.
Weather variables	Wind, gust, temperature, dewpoint/humidity, precipitation, cloud cover, pressure, visibility where available.
Derived features	launch wind fit, gust spread, cloudbase estimate, basic thermal potential, rain blocker, model confidence.
Map layers	Launch status icons, wind arrows/particles from sampled grid/API, rain/cloudbase/thermal simple overlays.
Reality layer	Pilot reports first; station ingest ready for club stations.
Validation	Daily forecast-vs-observation table for any sites with stations/reports.
AI briefing	Grounded summaries from site packs only; no raw weather hallucination.
Paid tier	PRO: alerts and 10-day forecast; PRO+: model comparison and advanced site confidence where available.

MVP acceptance tests:

- 100 launch sites can generate hourly forecast packs for 7-10 days.
- API returns site forecast in < 250ms cached.
- Missing wind/gust data blocks GO output.
- Alert engine detects a wind window and gust exceedance.
- AI briefing cites exact data values from site pack.
- Quality flags are visible in API and app.

27. V1/V2 roadmap

Phase	Weather capability	Business value
MVP	API-based site packs, basic flyability inputs, alerts.	Launch product and validate pilot demand.
V1	Raw GFS/ECMWF ingestion for priority regions, tile layers, station network.	Premium forecast differentiation and model comparison.
V1.5	Forecast validation dashboard, correction models per site, reality-vs-forecast map.	Trust and retention; club/school sales.
V2	Digital twin corrections, thermal model trained on tracks, terrain/rotor model, storm nowcasting.	Best-in-market paragliding decision intelligence.
V3	Global high-resolution regions, competition task weather center, rescue/weather reconstruction, API licensing.	Enterprise and ecosystem expansion.

- Priority regions: Norway, French/Swiss/Italian/Austrian Alps, Slovenia, Romania/Bulgaria mountain zones, Turkey, Spain/Pyrenees, Colombia and other high-volume flying locations.
- Priority premium data features: altitude wind, model disagreement, station correction, storm nowcast and site-specific bias correction.
- Priority trust features: show why forecast changed, show observed vs forecast, show quality flags, show confidence bands.

28. Engineering backlog

Epic	Tasks
WE-001 Provider abstraction	Define provider adapter interface; implement Open-Meteo and MET Norway adapters; add source license metadata.
WE-002 Model run manager	Create model_run lifecycle table, idempotency keys, status transitions and retry policy.
WE-003 Forecast cube MVP	Define canonical variables; store API forecasts as normalized site packs first; prepare Zarr metadata model.
WE-004 GRIB processing	Build ecCodes/cfgrib/xarray pipeline; normalize GFS sample; normalize ECMWF sample; unit tests.
WE-005 Site pack generator	Extract values, derive launch wind fit, gust spread, cloudbase estimate and quality flags.
WE-006 Terrain features	Import DEM, compute site slope/aspect/ridge/valley features, cache terrain profile.
WE-007 Derived paragliding features	Implement thermal v0, rotor v0, Foehn v0, valley wind v0, model disagreement v0.
WE-008 Tile service integration	Generate launch status vector layer and wind/thermal/risk tiles.
WE-009 Observation ingest	Station schema, pilot report ingest, station health checks, current reality cache.
WE-010 Forecast validation	Forecast error table, daily validation job, model skill dashboard.
WE-011 Correction engine	Bias correction baseline, site-specific correction with min sample rules.
WE-012 Alerts integration	Emit weather_window_changed events and current hazard events.
WE-013 Observability	Metrics, logs, run dashboard, quality flag dashboard, stale-data alert.
WE-014 Security/licensing	Secrets handling, attribution enforcement, entitlement checks for premium layers.

29. References

The following official and primary technical sources were used to ground the Weather Engine assumptions. These references should be reviewed again during implementation because provider availability, licensing, variables and rate limits can change.

- **ECMWF Open Data:** <https://www.ecmwf.int/en/forecasts/datasets/open-data> - ECMWF open forecast data and licensing/attribution basis.
- **ECMWF real-time open data access:** <https://www.ecmwf.int/en/forecasts/access-forecasts/access-real-time-open-data> - Access path for real-time open forecast products.
- **NOAA NOMADS:** <https://nomads.ncep.noaa.gov/> - NOAA/NCEP model data access portal including GFS products.
- **NCEP GRIB2 Documentation:** https://www.nco.ncep.noaa.gov/pmb/docs/grib2/grib2_doc/ - GRIB2 documentation for NCEP products.
- **MET Norway Locationforecast:** <https://api.met.no/weatherapi/locationforecast/2.0/documentation> - Point forecast service documentation and limitations.
- **MET Norway Weather API:** <https://api.met.no/> - MET Norway API terms, User-Agent guidance and licensing context.
- **Open-Meteo Forecast API:** <https://open-meteo.com/en/docs> - MVP forecast API and variable/model access.
- **Xarray GRIB via cfgrib:** <https://docs.xarray.dev/en/stable/user-guide/io.html#grib-format-via-cfgrib> - Python processing of GRIB with xarray and cfgrib.
- **Copernicus DEM:** <https://dataspace.copernicus.eu/explore-data/data-collections/copernicus-contributing-missions/collections-description/COP-DEM> - Digital elevation model for terrain processing.
- **OpenAIP API Documentation:** <https://docs.openaip.net/> - Aeronautical/airspace data API for flight safety layers.
- **OGC API - Tiles:** <https://ogcapi.ogc.org/tiles/> - Standards context for tiled geospatial APIs.

Appendix A - Weather Engine SQL seeds

```
CREATE SCHEMA IF NOT EXISTS weather;
```

```
CREATE TABLE weather.sources (  
  id UUID PRIMARY KEY,  
  name TEXT NOT NULL,  
  provider_type TEXT NOT NULL,  
  license TEXT NOT NULL,  
  attribution_text TEXT NOT NULL,  
  commercial_use_allowed BOOLEAN NOT NULL DEFAULT false,  
  redistribution_allowed BOOLEAN NOT NULL DEFAULT false,  
  status TEXT NOT NULL DEFAULT 'active',  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

```
CREATE TABLE weather.model_runs (  
  id UUID PRIMARY KEY,  
  source_id UUID NOT NULL REFERENCES weather.sources(id),  
  model_name TEXT NOT NULL,  
  run_time_utc TIMESTAMPTZ NOT NULL,  
  cycle TEXT NOT NULL,  
  forecast_horizon_hours INT NOT NULL,  
  status TEXT NOT NULL,  
  raw_manifest_uri TEXT,  
  manifest_hash TEXT,  
  processor_version TEXT,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  UNIQUE(source_id, model_name, run_time_utc, cycle)  
);
```

```
CREATE TABLE weather.raw_files (  
  id UUID PRIMARY KEY,  
  model_run_id UUID NOT NULL REFERENCES weather.model_runs(id),  
  object_uri TEXT NOT NULL,  
  variable_group TEXT NOT NULL,  
  forecast_step_hours INT,  
  checksum_sha256 TEXT,  
  size_bytes BIGINT,  
  validation_status TEXT NOT NULL DEFAULT 'pending',  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

```
CREATE TABLE weather.forecast_cubes (  
  id UUID PRIMARY KEY,  
  model_run_id UUID NOT NULL REFERENCES weather.model_runs(id),  
  variable TEXT NOT NULL,  
  level_type TEXT NOT NULL,  
  level_value NUMERIC,  
  units TEXT NOT NULL,  
  storage_uri TEXT NOT NULL,  
  valid_from_utc TIMESTAMPTZ NOT NULL,  
  valid_to_utc TIMESTAMPTZ NOT NULL,  
  normalization_version TEXT NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

```
CREATE TABLE weather.site_forecast_packs (  
  site_id UUID NOT NULL,  
  valid_time_utc TIMESTAMPTZ NOT NULL,  
  model_run_id UUID NOT NULL REFERENCES weather.model_runs(id),  
  corrected_version TEXT,  
  wind JSONB NOT NULL,  
  cloudbase JSONB,  
  thermal JSONB,  
  precip_storm JSONB,  
  terrain_risk JSONB,  
  model_agreement JSONB,  
  quality_flags JSONB NOT NULL DEFAULT '[]::jsonb',  
  algorithm_versions JSONB NOT NULL,  
  generated_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  PRIMARY KEY(site_id, valid_time_utc, model_run_id)  
);
```

```
CREATE INDEX idx_site_forecast_site_time  
  ON weather.site_forecast_packs(site_id, valid_time_utc DESC);
```

```
CREATE TABLE weather.forecast_error_samples (  
  id UUID PRIMARY KEY,  
  site_id UUID NOT NULL,  
  model_run_id UUID REFERENCES weather.model_runs(id),  
  variable TEXT NOT NULL,  
  horizon_hours INT NOT NULL,  
  forecast_value NUMERIC,  
  observed_value NUMERIC,  
  error_value NUMERIC,  
  observed_at TIMESTAMPTZ NOT NULL,  
  group_key JSONB NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

Appendix B - Example derived feature code contract

```
class DerivedFeatureWorker:  
  name = "thermal-potential-v1"  
  required_variables = [  
    "temperature", "dewpoint", "total_cloud_cover",  
    "shortwave_radiation", "u_wind", "v_wind"  
  ]  
  
  def compute(self, site, valid_time, raw_values, terrain_profile):  
    missing = self.find_missing(raw_values)  
    if missing:  
      return FeatureResult(value=None, quality_flags=[f"MISSING:{v}" for v in missing])  
  
    radiation_factor = clamp(raw_values.shortwave_radiation / 850.0, 0, 1)  
    cloud_penalty = clamp(raw_values.total_cloud_cover / 100.0, 0, 1)  
    slope_bonus = terrain_profile.sun_facing_slope_score(valid_time)  
    wind_penalty = self.wind_penalty(raw_values.wind_launch_kmh)
```

```
potential_ms = 4.0 * radiation_factor * (1 - 0.65 * cloud_penalty) * slope_bonus * wind_penalty  
confidence = self.confidence(raw_values, terrain_profile)  
return FeatureResult(value=potential_ms, confidence=confidence, quality_flags=[])
```

End of Volume 05

The Weather Engine Design is the technical foundation for raw forecasts, site-level decision support, premium model comparison, nowcasting, validation, digital twin correction, and map/weather layers. The next volume should be Volume 06 - AI & ML Architecture, which depends on the data products defined here.